

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №9
По дисциплине МПИС
Тема: «Списки. Создание собственного адаптера. Механизмы обратного
вызова.»

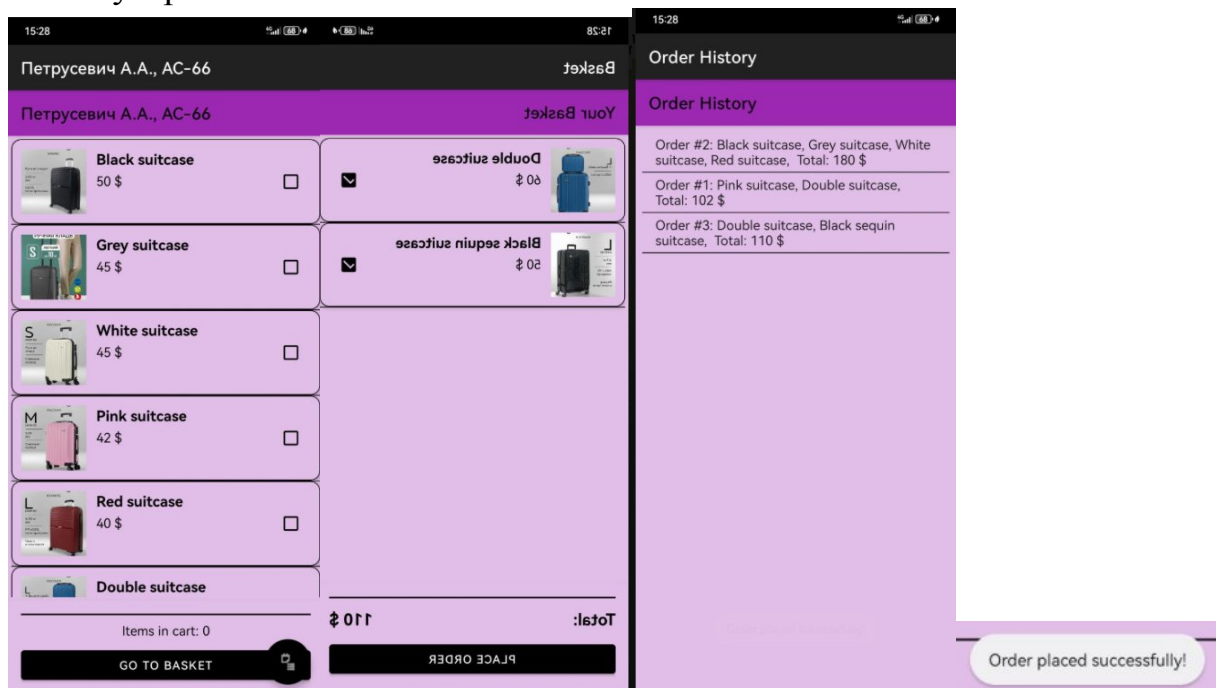
Выполнила:
Студентка 3-го курса
Группы АС-66
Петрусевич А.А.
Проверил:
Козинский А. А.

Брест 2026

Цель работы: Разработать приложение MiniShop, состоящее из двух Activity.

Практическое задание

1. Разработать приложение MiniShop, состоящее из двух Activity (см. рисунки 3.3, 3.4 в источнике).
2. В первом Activity создать список ListView с Header и Footer.
3. В Footer разместить текстовое поле (TextView) для ввода количества активированных пользователем товаров, кнопку Show Checked Items для перехода в корзину товаров.
4. Реализовать кастомизированный список ListView с помощью собственного адаптера, наследующего класс BaseAdapter.
5. В каждом пункте списка отобразить следующую информацию о товаре: идентификационный номер, название, стоимость, чекбокс для возможности выбора товара пользователем.
6. В текстовом поле (TextView) Footer списка динамически отображать общее текущее количество активированных товаров.
7. При нажатии кнопки Show Checked Items реализовать переход во второе Activity с корзиной товаров.
8. Корзину товаров реализовать в виде нового кастомизированного списка с выбранными товарами.
9. Продемонстрировать работу приложения MiniShop на эмуляторе или реальном устройстве



Вывод и выполненная работа:

В процессе разработки приложения MiniShop были использованы следующие API и компоненты Android SDK:

Для организации навигации между двумя Activity применялся механизм явных Intent с передачей данных через `putExtra()` и `getSerializableExtra()` для передачи коллекции выбранных товаров в корзину.

Основным элементом интерфейса первого Activity выступал `ListView`, для которого были реализованы `addHeaderView()` и `addFooterView()` для добавления заголовка и подвала списка.

Для отображения списка товаров был создан кастомизированный адаптер, наследующий `BaseAdapter`, с переопределением методов `getCount()`, `getItem()`, `getItemId()` и `getView()`, что позволило сформировать индивидуальный макет для каждого элемента списка. В `getView()` использовался паттерн `ViewHolder` для оптимизации производительности при прокрутке.

Каждый элемент списка содержал компоненты `TextView` для вывода идентификационного номера, названия и стоимости товара, а также `CheckBox` для выбора товара пользователем.

Для отслеживания изменений состояния чекбоксов применялся интерфейс `OnCheckedChangeListener`, а общее количество выбранных товаров динамически обновлялось в `TextView`, расположенном в Footer списка.

Переход во второе Activity осуществлялся по нажатию кнопки `Button` с использованием `setOnClickListener()`, при этом формировался `ArrayList` с объектами выбранных товаров, который передавался через `Intent`.

Во втором Activity корзина товаров реализовывалась с помощью еще одного кастомизированного `ListView` на базе `BaseAdapter`, отображающего только те товары, которые были выбраны пользователем в первом экране.

Для передачи данных между Activity использовался механизм `Serializable` или `Parcelable` для объектов товаров.

Все эти компоненты в совокупности позволили создать двухэкранное приложение с кастомизированными списками, динамическим подсчетом выбранных элементов и передачей данных между экранами без использования сторонних API, только на основе стандартного Android SDK.